

CAREBOT

Caregiver Connect — Crowdsourcing Team

Sprint 3 Presentation | CS 495 Senior Capstone | University of Alabama



Cole Segura



Carlos Marquez



Sheala Miller



Jaylon Sanders



Jarl Evanson

GitHub: github.com/Distributed-Autonomy-Lab/carebot-deployable-crowdsourcing

Website: <https://distributed-autonomy-lab.github.io/crowdsourcing-senior-info-website/>

Cloud Deployment Website: <https://34.70.170.87/>

April 23, 2026

PROJECT FOCUS

What is Carebot and what are we building?

The Application

Django web app backed by PostgreSQL, PostGIS, and pgvector that helps Alabama residents find healthcare resources through a Google Gemini AI chatbot

Stack: Django 5.0, PostgreSQL 17, PostGIS, pgvector, Sentence Transformers, Gemini, Docker Compose



Sprint 3 Focus

- User Guide: end-user documentation for caregivers
- Accessibility Research: WCAG 2.1 AA rationale, cited sources
- Production hardening: GCP deployment fixes
- Similarity search tuning and retest tooling



Our Focus

- Enable community resource submissions
- Build moderation workflow
- Implement user accessibility settings
- Improve development environment & security

SPRINT 3 GOALS

Prioritized objectives for Mar 25 – Apr 21, 2026



End-User Documentation



Accessibility Research Writeup



Production Hardening



Similarity Search Tuning



Test Plan Execution

MAJOR ACHIEVEMENTS



Documentation Pack

- Caregiver user guide and sponsor user guide
- Development guide, run & deploy guide, environment setup
- Test plan & FAQs



Accessibility & UI

- Django template passes on public submit and moderator flows
- Contrast, touch targets, labels, semantics aligned to WCAG 2.1 AA
- Accessibility research writeup backing the design choices



Deployment & Security

- Google Cloud deployment templates and env examples
- Cybersecurity assessment



Testing & Moderation

- End-to-end and integration tests
- Unit tests for submission and merge flows
- Moderation and dashboard UX improvements

SPRINT 3 BACKLOG

Purpose: Course finish + handoff

- **Documentation pack:** user guide, accessibility research, development guide, deploy/run, FAQs, test plan, sponsor guide, final doc hub.
- **Accessibility & UI:** template/static work for public + moderator flows (WCAG-oriented)
- **Deploy / GCloud path:** docker-compose.gcloud.yml, env examples, collectstatic / entrypoint behavior as documented.
- **Final course artifacts:** demo video + cybersecurity assessment
- **Debugging and Testing**

ID	Item	Priority	Status
S3.2	Accessibility & inclusive UI — Django templates/static (public + moderator flows)	P0	Done / updated
S3.D1	Documentation — caregiver user guide + accessibility research	P0	Done
S3.D2	Documentation — development guide (stack, repo, workflows)	P0	Done
S3.D3	Documentation — run & deploy + development environment (Docker, keys)	P0	Done / updated
S3.D4	Documentation — FAQs (APIs, OAuth, gotchas)	P0	Done
S3.D5	Documentation — test plan (cases + scripts)	P0	Done / updated
S3.D6	Documentation — sponsor user guide (stakeholder flows)	P0	Done / updated
S3.3	Deployment hardening & environment templates (app repo + cloud examples)	P1	Done / updated
S3.4	Final deliverables — demo video + cybersecurity assessment	P0	In progress / submit
S3.5	Moderation/dashboard UX (layout, reporting) where merged in app	P1	Done / partial
S3.1	Bulk moderation (multi-select) for moderation queue	P2	Not done

PROJECT BACKLOG

What we built	Sprint 1	Sprint 2	Sprint 3
Public submissions — form, validation, caregiver-facing flow	Sprint 1 — fixed broken submit path; rate limits, headers, basic hardening so public POST was safe.	Sprint 2 — new submit UI, home/base templates, working user/staff login before moderators touch the queue.	Sprint 3 — accessibility and template polish on the same flows (labels, contrast, targets).
Moderation queue — approve · merge · reject (single item)	Sprint 1 — shipped the queue and all three actions end-to-end; first usable moderator loop.	Sprint 2 — queue HTML/CSS and Django wiring for the redesigned experience.	Sprint 3 — moderator screens easier to use with a keyboard and screen reader. Bulk multi-select not implemented —still one row at a time.
Merge & deduplication — field-level smart merge + similarity hints	Sprint 1 — field-level merge into an existing resource; reject and “approve as new”; core dedup story.	Sprint 2 — moderator-facing similarity UX so merges are informed by scores, not guesswork.	Sprint 3 — small hardening and doc alignment with how merge behaves in production.

PROJECT BACKLOG

Similarity search — TF-IDF, embeddings, pgvector	Sprint 1 — first similarity pass for “near duplicate” candidates on the moderation path.	Sprint 2 — main build: full-catalog TF-IDF + pgvector, rank_similar_smart, parameterized SQL, moderator dedup tooling.	Sprint 3 — operational notes, test/deploy docs; no second large algorithm rewrite.
Docker & deploy — local + cloud demo	Sprint 1 — Docker on Mac and Windows; baseline compose for the class.	Sprint 2 — GCP VM demo (firewall, secrets, container on the network).	Sprint 3 — env templates and run/deploy guidance for a repeatable handoff.
Documentation — runbooks + this site	CHANGES / build notes so teammates could boot the stack.	README, similarity/schema runbooks in the app repo.	Final GitHub Pages hub (guides, FAQs, test plan, security write-up) this page plus linked artifacts.

Technologies Used and Experience



Technologies Used

- Django
- PostgreSQL with PostGIS & pgvector extensions
- Docker & Docker Compose, Gunicorn, NGINX
- Sentence Transformers & TF-IDF
- GCP Compute Engine



Django & Similarity

Django

- Well-documented & easy to deploy
- Modular architecture made modification simple

Similarity

- Easy to use; understandable implementation
- Reliable and intelligent results



Postgres + Django Migrations

- Frustrating
- Non-idiomatic usage caused persistent issues
- Frequent migration failures



Local & Cloud Deployment

Docker

- Initially challenging to use
- Became highly beneficial after cleanup

GCP & GCP Compute Engine

- Well-documented for our use case
- Minimal challenges during deployment

NOT COMPLETED & LESSONS LEARNED



Deferred Beyond Sprint 3

- Bulk moderation (approve / reject multiple at once)
- Email and SMS notifications on submission status
- Provider verification workflow
- Admin dashboard with analytics
- Chatbot integration for approved submissions
- Accessibility QA with real older-adult users



Lessons Learned

- Design-first work from Sprint 2 made Sprint 3 documentation faster because the UI was stable
- Writing docs against shipped code catches drift between intention and implementation
- Research-backed writing is slower but worth it for the final deliverable

Accessibility and Inclusive Design Features



Text Size & Text-to-Speech

- Three text size options, 14pt, 16pt, & 20pt
- “Read Page” Button for user-controlled web page reading



Color Contrast Modes

- Default (Light Mode)
- Dark Mode: High and Low Contrast
- Red Color Blind
- Red/Green Color Blind
- Blue/Green Color Blind



Focus Highlight

- Toggleable
- Visibility of focus outlines on form



Form Design

- Large Visible Buttons
- Logical Section Split
- Simple & Intuitive Layout

TEAM CONTRIBUTIONS



Cole Segura

Accessibility Design & Research writeup, User Guide for the public submission flow



Carlos Marquez

User Accessibility Setting Implementation, Development Guide



Sheala Miller

Github Website, scrum coordination, FAQs and Gotchas



Jaylon Sanders

Test plan execution and accessibility QA, Test Plan and Test Cases



Jarl Evanson

Backend and development/deployment configuration cleanup, Cloud Deployment Guide

Conclusion